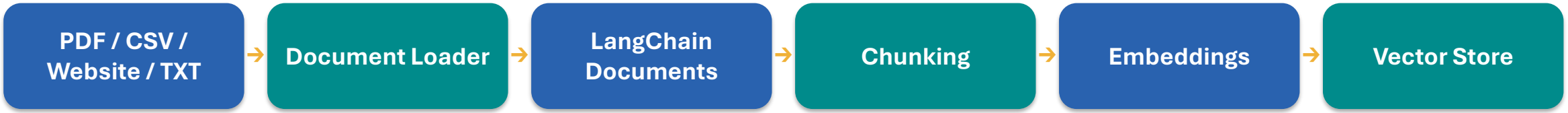
1. What Is a Document Loader?

DOCUMENT LOADING

- A Document Loader takes data from different sources and converts it into a format LangChain can process.
- Simple definition: a Document Loader brings external data into a LangChain application.
- A document commonly contains page_content (the extracted text) and metadata (information about its source).
- Example metadata: source = notes.pdf; page = 2.
- The LLM does not automatically read a PDF just because it exists on your computer; the loader extracts content first.

Document Loading Pipeline

DOCUMENT LOADING



- The loaded documents can be split into chunks, embedded, stored, and later retrieved for RAG.

2. PDF Loaders

DOCUMENT LOADING

- A PDF Loader extracts text from PDF files.
- Example file: GenAI_Notes.pdf, containing pages about LLMs, Transformers, RAG, and LangChain.
- The loader extracts page content and can preserve page-related metadata.
- Why it matters: it avoids manually copying content from long documents.
- Classroom example: load a 500-page company policy PDF so a RAG system can search it.

PDF Loader: Python Example

DOCUMENT LOADING

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("genai.pdf")
documents = loader.load()

# documents contains the extracted pages
print(documents[0].page_content)
print(documents[0].metadata)
```

- The loader's output is a list of Document objects.

3. CSV Loaders

DOCUMENT LOADING

- CSV means Comma-Separated Values.
- Example columns: Name, Age, Department.
- Example rows: Rahul, 25, IT; Priya, 27, HR; Amit, 24, Finance.
- A CSV loader reads tabular data and converts rows into documents.
- Real-world example: products.csv with product, price, and rating; a user asks which product has a rating above 4.3.

CSV Loader: Python Example

DOCUMENT LOADING

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader(file_path="employees.csv")
documents = loader.load()

print(documents[0].page_content)
print(documents[0].metadata)
```

- The exact row formatting depends on the loader configuration and source CSV.

4. Web Loaders

DOCUMENT LOADING

- Web loaders extract content from webpages.
- They can retrieve webpage text so the content can be processed by a LangChain application.
- Example source: a company or college website.
- Real-world application: a college admission chatbot that answers questions about courses, fees, admission, exams, and hostels.
- Use responsibly and respect site access rules, robots policies, and terms of service.

Web Loader: Python Example

DOCUMENT LOADING

```
from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader("https://example.com")
documents = loader.load()

print(documents[0].page_content)
```

- Real websites may require configuration and may contain content that is not available as plain HTML.

5. Text Loaders

DOCUMENT LOADING

- Text loaders load ordinary text files, such as notes.txt.
- Example content: “LangChain is a framework for building applications powered by language models.”
- The loader reads the text and wraps it as a LangChain Document.
- Useful for notes, logs, transcripts, and simple text-based knowledge sources.

Text Loader: Python Example

DOCUMENT LOADING

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("notes.txt")
documents = loader.load()

print(documents[0].page_content)
```

Loader Comparison

DOCUMENT LOADING

- PDF Loader → PDF files
- CSV Loader → CSV files / tabular rows
- Web Loader → webpage content
- Text Loader → TXT files
- All loaders help convert source data into LangChain Documents.

6. Why Do We Need Text Splitters?

TEXT SPLITTING

- A very large document may not fit within the model's context window or may be inefficient to send all at once.
- Text splitting (chunking) divides a document into smaller pieces.
- Reasons: context limits, better retrieval granularity, and more useful embedding/search units.
- Pipeline: Large Document → Text Splitter → Chunk 1, Chunk 2, Chunk 3, ...

7. Character Splitter

TEXT SPLITTING

- A Character Splitter divides text based mainly on a fixed character size.
- Example: a 10,000-character document can be divided into chunks of around 1,000 characters.
- chunk_size controls the target chunk size; chunk_overlap repeats a small portion across neighboring chunks.
- Overlap helps preserve context when a useful sentence or idea crosses a chunk boundary.

Character Splitter: Python Example

TEXT SPLITTING

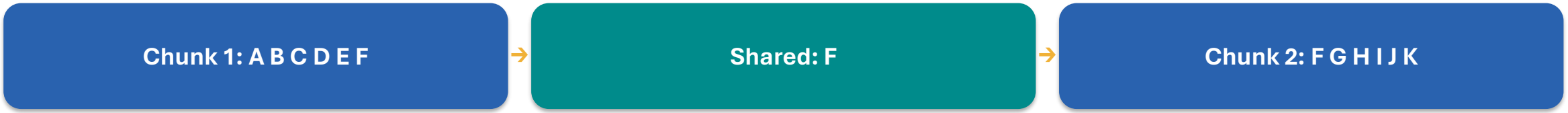
```
from langchain_text_splitters import CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)

chunks = splitter.split_documents(documents)
```

What Is Chunk Overlap?

TEXT SPLITTING



- A portion of text appears in both neighboring chunks.
- This reduces the chance that a boundary separates a key phrase from its surrounding explanation.

8. Recursive Character Splitter

TEXT SPLITTING

- The Recursive Character Splitter tries different separators to split text at natural boundaries.
- It may try paragraph breaks, then line breaks, then sentence/word boundaries, and eventually smaller units if needed.
- It aims to respect text structure better than blindly cutting at an arbitrary position.
- Simple comparison: character splitter says “cut by size”; recursive splitter says “try natural boundaries while respecting the target size.”

Recursive Splitter: Python Example

TEXT SPLITTING

```
        from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_documents(documents)
```

9. Semantic Splitter

TEXT SPLITTING

- A Semantic Splitter divides text according to meaning or topic changes, not just character count.
- It can use embeddings or semantic similarity to estimate where one idea ends and another begins.
- Example topics: Python programming; Delhi weather; Delhi historical monuments.
- A semantic splitter attempts to group related sentences and split when the meaning changes.
- Its boundaries depend on the implementation and thresholds; it is not guaranteed to produce perfect topics.

Three Splitter Types: Compare

TEXT SPLITTING

- Character Splitter → mainly size-based.
- Recursive Splitter → structure-aware; tries hierarchical separators.
- Semantic Splitter → meaning/topic-aware.
- Choose based on the data, retrieval needs, compute cost, and acceptable chunk sizes.

10. Retriever Pipelines

RETRIEVAL STRATEGIES

- A retriever finds information relevant to the user's question.
- Typical flow: User Question → Retriever → Relevant Documents → Context → LLM → Answer.
- Example: “What is the company’s refund policy?” retrieves the relevant chunk from Refund Policy.pdf.
- A retriever usually returns documents; the LLM uses them to formulate the final answer.

11. Similarity Retrieval

RETRIEVAL STRATEGIES

- Similarity retrieval finds documents whose embeddings are closest to the query embedding.
- The question is embedded, compared with stored document embeddings, and the highest-scoring matches are returned.
- Illustrative scores: Document 1 = 0.91; Document 2 = 0.83; Document 3 = 0.42.
- Simple definition: it finds documents that are mathematically close in meaning to the user's question.
- Similarity scores are not always probabilities; their interpretation depends on the embedding model and metric.

12. MMR Retrieval

RETRIEVAL STRATEGIES

- MMR means Maximal Marginal Relevance.
- It balances relevance to the query with diversity among selected results.
- Similarity-only search might return three nearly identical refund-policy chunks.
- MMR tries to return relevant but less redundant material, such as refund policy, shipping policy, and exceptions when useful.
- Trainer line: Similarity asks “Which documents are most similar?” MMR asks “Which documents are relevant and add different information?”

Similarity vs. MMR

RETRIEVAL STRATEGIES

- Similarity retrieval: focuses on relevance; may return highly similar or redundant results.
- MMR retrieval: balances relevance and diversity; aims to reduce redundancy.
- Similarity is useful for direct matching.
- MMR is useful when the answer benefits from a range of relevant perspectives or details.

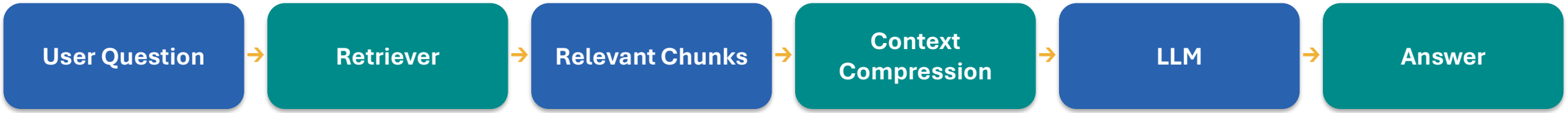
13. Context Compression

RETRIEVAL STRATEGIES

- Retrieval may return more text than the question needs.
- Context compression filters or condenses retrieved documents to retain the most useful information.
- Example: for “What is the refund period?”, keep the refund rule and remove unrelated company history, products, and contact information.
- Benefits: less token usage, less noise, potentially lower cost, and better context quality.
- Compression must preserve key facts and conditions; over-compression can remove important qualifications.

Retriever Pipeline: End-to-End

RETRIEVAL STRATEGIES



- Similarity or MMR can determine which documents are selected; compression can refine the selected context before generation.

14. Conversation Memory

MEMORY

- Conversation memory makes previous conversation information available to later turns.
- Example: “My name is Rahul.” Later: “What is my name?”
- If the earlier message or an appropriate memory record is supplied, the assistant can answer “Rahul.”
- A basic flow is: User Message → History/Memory → LLM → Answer → Update History.
- Important: memory is an application feature; an LLM does not automatically remember all prior calls.

15. Summary Memory

MEMORY

- Long conversation histories can become too large to include in full.
- Summary memory keeps a compact summary of important details rather than every message.
- Example summary: “User is learning GenAI and is interested in RAG and Transformers.”
- Flow: Long Conversation → Summarization → Compact Summary → LLM.
- Advantage: reduces context use; limitation: a summary may omit details.

16. Vector Memory

MEMORY

- Vector memory stores information as embeddings in a vector store.
- Later, a query can retrieve semantically related memories.
- Example: earlier memory says “I am learning LangChain and RAG”; later question: “What framework was I studying?”
- Flow: Conversation/Fact → Embedding → Vector Store → Similarity Retrieval → Relevant Memory.
- It is useful for retrieving older, related information that may not be in the recent chat window.

Three Memory Approaches: Compare

MEMORY

- Conversation Memory → keeps conversation history available.
- Summary Memory → stores a compressed representation of the conversation.
- Vector Memory → retrieves memories by semantic similarity.
- These approaches can be combined; each has trade-offs in detail, token usage, retrieval quality, and implementation complexity.

17. What Are Tools?

TOOLS & CHAINS

- A tool lets an LLM-powered application perform an external action or call a capability.
- Examples: calculator, search, database, API, or a custom Python function.
- A model may decide a tool is needed; the application executes the tool and returns its result to the model.
- Flow: User Question → LLM/Agent → Tool → External Result → LLM → Answer.

18. Calculator Tool

TOOLS & CHAINS

- A calculator tool performs arithmetic using a dedicated computation function.
- Example question: “What is 345 × 678?”
- The model can call the calculator, receive the result, and explain it in a natural-language answer.
- This is useful for arithmetic where a language model's text generation alone may be unreliable.

19. Search Tool

TOOLS & CHAINS

- A search tool retrieves information from a search system or other configured source.
- Example: “What happened in today's technology news?”
- Flow: Question → LLM/Agent → Search Tool → Search Results → LLM → Answer.
- Freshness depends on the search provider, query, access, and source quality.

20. Custom Tools

TOOLS & CHAINS

- A custom tool is a function or capability created by the developer and exposed to the LLM/agent.
- Examples: get_weather(), check_order_status(), calculate_tax(), get_customer_details(), book_ticket().
- Example function: get_employee_salary(employee_id).
- The agent can select the tool when the request requires that capability.
- Apply authorization and input validation, especially for sensitive actions such as payments or customer records.

21. Sequential Chains

TOOLS & CHAINS

- A sequential chain runs steps one after another.
- The output of one step becomes the input to the next step.
- Example: Generate Article → Summarize Article → Translate Summary.
- Use sequential workflows when a later step depends on an earlier result.

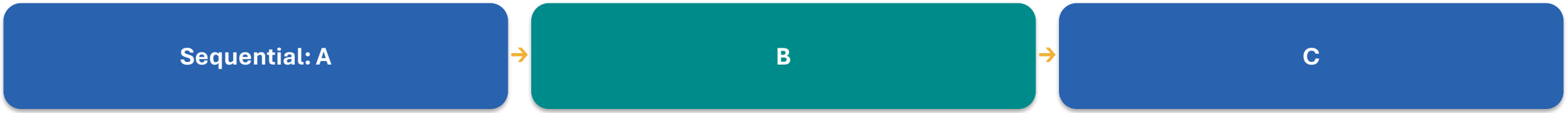
22. Parallel Chains

TOOLS & CHAINS

- Parallel chains run independent tasks at the same time when the framework and runtime support it.
- Example: from one input, generate a summary, extract keywords, and classify sentiment.
- The outputs can be collected into one result.
- Benefit: potentially lower latency for independent tasks; overall time depends on runtime and task behavior.

Sequential vs. Parallel

TOOLS & CHAINS



- Sequential: A → B → C; each step follows the previous step.
- Parallel: input branches to A, B, and C; independent tasks run together.
- Choose sequential when there are dependencies; choose parallel when tasks are independent.

23. LCEL Basics

LCEL & RUNNABLE WORKFLOWS

- LCEL means LangChain Expression Language.
- It provides a composable way to connect LangChain components into workflows.
- A recognizable feature is the pipe operator: |
- Example: prompt | model | parser.
- Conceptual flow: Input → Prompt → LLM → Parser → Output.
- Trainer line: LCEL lets us connect components like building blocks.

LCEL: Python Example


```
chain = prompt | model | parser

result = chain.invoke({"topic": "RAG"})
```

- Here, prompt formats the input, model generates output, and parser converts it to the desired form.

25. Structured Output Design

STRUCTURED OUTPUT & PYDANTIC

- Structured output means asking the model to return data in a defined shape rather than unrestricted prose.
- Free-form example: “The candidate is Ishita. She has 2 years of experience. Her skills include Python and SQL.”
- Structured example: an object with name, experience, and skills fields.
- Structured data is easier for applications to validate, store, query, and pass to other software components.

26. Why Structured Output?

- Imagine a recruitment application needs Name, Email, Skills, Experience, and Education.
- Free-form text is harder to process consistently.
- A defined schema tells the model and application which fields are expected.
- Example JSON fields: name (string), email (string), skills (list), experience (number).
- A schema improves consistency, but the result should still be validated.

Structured Output: Example


```
                                                {
  "name": "Ishita",
  "experience": 2,
  "skills": ["Python", "SQL"]
}
```

- This is illustrative JSON: it represents fields an application can process directly.

27. Pydantic Parsers

STRUCTURED OUTPUT & PYDANTIC

- Pydantic lets developers define expected data structures with Python classes and types.
- A Pydantic model describes fields such as name, age, and course.
- It validates incoming data against the schema and reports mismatches.
- In LangChain, Pydantic-based output parsers can parse model output into validated structured objects.

Pydantic: Python Example


```
from pydantic import BaseModel

class Student(BaseModel):
    name: str
    age: int
    course: str

student = Student(
    name="Rahul",
    age=22,
    course="GenAI"
)

print(student.name)
```

28. Why Pydantic Is Useful with LLMs

STRUCTURED OUTPUT & PYDANTIC



- LLMs generate probabilistic output; applications need predictable fields and types.
- Pydantic acts as a validation layer.
- If age must be an integer but the output is “twenty two”, validation/coercion rules determine whether it can be accepted; strictness can be configured.

31. Most Important Differences: Splitters

REVISION

- Character Splitter: mainly size-based.
- Recursive Splitter: uses hierarchical separators to preserve structure where possible.
- Semantic Splitter: uses meaning/topic changes to decide boundaries.
- Quick recall: size-based vs. structure-aware vs. meaning-based.

31. Most Important Differences: Retrieval

REVISION

- Similarity Retrieval: emphasizes relevance to the query.
- MMR Retrieval: balances relevance with diversity to reduce redundant results.
- Context Compression: reduces retrieved content to the useful information for the question.
- These techniques can work together in a retrieval pipeline.

31. Most Important Differences: Memory

REVISION

- Conversation Memory → conversation history.
- Summary Memory → compressed conversation summary.
- Vector Memory → semantic retrieval over stored memories.
- Memory designs can be combined depending on whether recency, compactness, or semantic recall matters most.

31. Free-Form vs. Structured Output

REVISION

- Free-form: “The student is Rahul and he is 22 years old.”
- Structured: { name: "Rahul", age: 22 }
- Structured output is easier for software to consume.
- Pydantic validates the shape and types of the data before use.